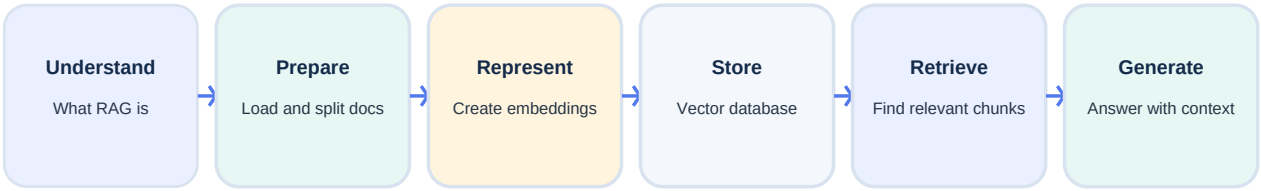
RETRIEVAL-AUGMENTED GENERATION (RAG)

Flow-wise notes • concepts, pipeline cards, and revision sheet

Core idea

Retrieve relevant information from your own documents, then give that context to an LLM so it can generate a grounded answer.

WHAT THESE NOTES COVER

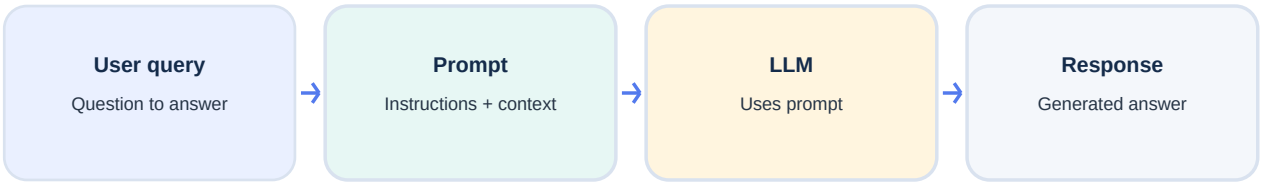


Built from the supplied RAG PDF. The source also introduces in-context learning and an example of an instruction that asks the model to answer only from supplied context.

1. Foundation: Context, ICL & RAG

In-context learning (ICL)

In-context learning is a capability of large language models (LLMs) in which the model uses examples or information placed in the prompt to guide its response. The model is not necessarily being retrained; the prompt supplies the immediate context.



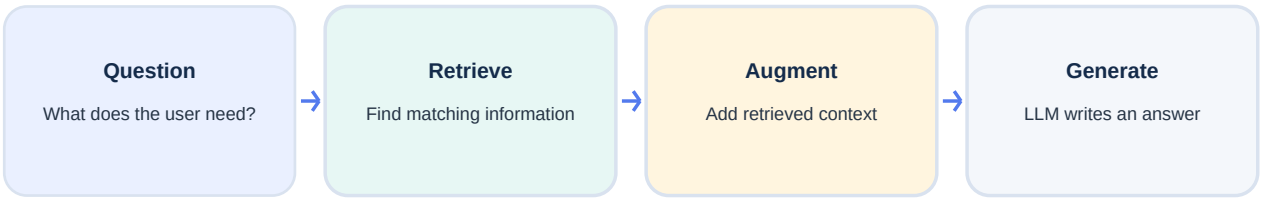
The supplied notes show a context-aware prompt pattern:

“You are a helpful assistant. Answer the question ONLY from the provided context. If the context is insufficient, just say you don’t know.”

```
Context: {context}
Question: {question}
```

What is RAG?

Retrieval-Augmented Generation combines information retrieval with text generation. Instead of asking the LLM to rely only on what it learned during training, the system retrieves relevant pieces of external information and supplies them as context for the answer.



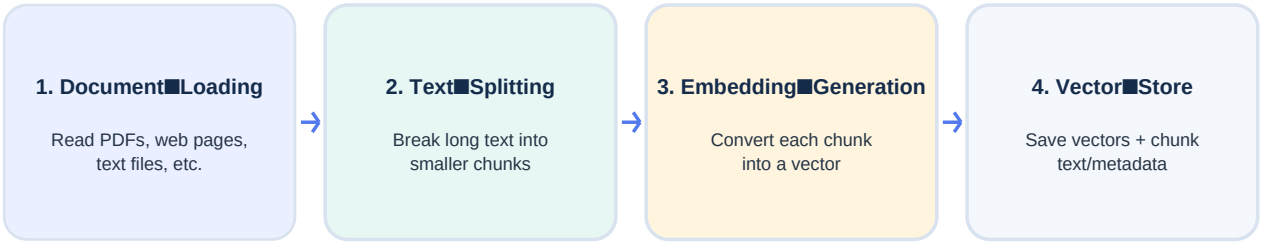
Remember: Retrieval finds evidence; augmentation puts that evidence into the prompt; generation turns the question plus evidence into a natural-language answer.

RAG vs. plain prompting

Plain prompting	RAG prompting
LLM answers from its existing learned patterns and whatever is written in the prompt.	LLM receives retrieved document chunks in addition to the user’s question.
May lack access to private, current, or domain-specific documents.	Can ground the answer in a supplied knowledge source; quality depends on retrieval and source quality.

2. RAG Ingestion Pipeline (Prepare the Knowledge)

This is the preparation stage. It is usually run before users ask questions, and it turns raw documents into searchable chunks.



Step 1 — Document loading

A document loader reads source material and returns document objects containing text and metadata. The source PDF illustrates a web/document source being loaded into a document representation.

• **Input:** raw source documents (for example, PDF, web page, or text).

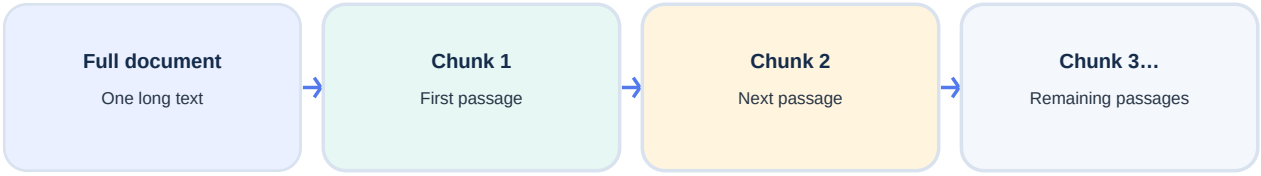
• **Output:** document text, often with metadata such as the source or page.

Step 2 — Text splitting / chunking

Long documents are divided into smaller, meaningful pieces called **chunks**. Chunking helps retrieval return focused passages rather than an entire large document.

• Chunks that are too large may contain unrelated information and consume more prompt space.

• Chunks that are too small may lose context. Choose chunk size and overlap based on the documents and task.



Step 3 — Embedding generation

An embedding model converts each text chunk into a numerical vector (a list of numbers) that represents semantic meaning. Similar meanings tend to have vectors that are close under the chosen similarity measure.

• **Text chunk → embedding model → vector.**

• The vector is a searchable representation; keep the original chunk text too, because the LLM needs readable context.

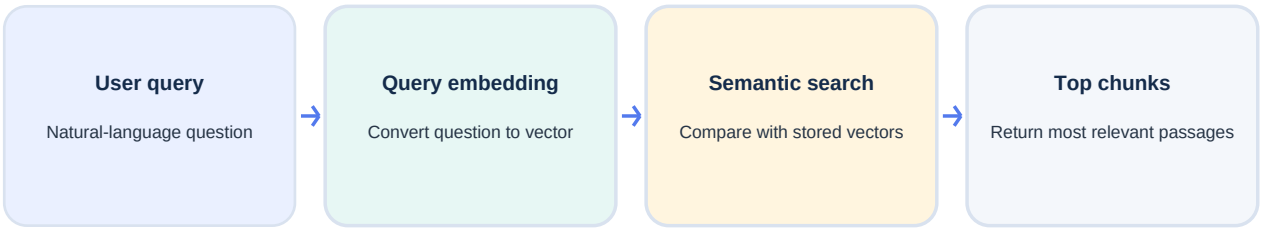
Step 4 — Store in a vector store

Store the embedding vectors along with their associated chunk text and useful metadata. At query time, the system searches this index for chunks that match the question semantically.

Ingestion output: a searchable collection of chunk embeddings linked to the original text.

3. Query-Time Pipeline: Retrieval

Retrieval is the real-time process of finding the most relevant pieces of information from the knowledge base for the user's query.



Flow explained

1. Receive the question. Example: "What does the policy say about refunds?"

2. Embed the question. Use the compatible embedding model to convert the query into a vector.

3. Search the vector store. Compare the query vector with stored document vectors using a similarity method.

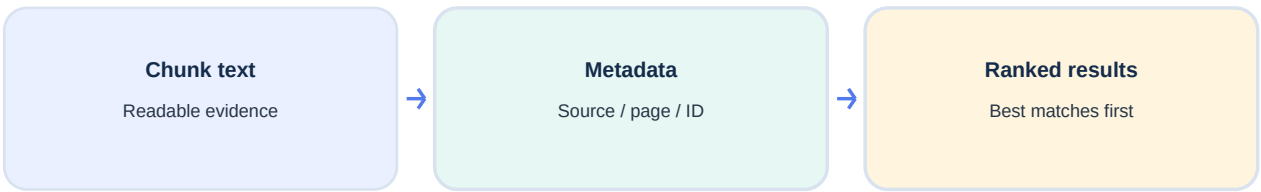
4. Return relevant chunks. Retrieve the best-matching passages and their source metadata. The retrieved text becomes the evidence/context for the next stage.

Semantic search vs. keyword search

Keyword-style match	Semantic search
Often looks for exact or overlapping terms.	Uses embeddings to find related meaning, even when wording differs.
May miss a passage that expresses the idea differently.	Can surface conceptually relevant passages; it can still retrieve the wrong chunks.

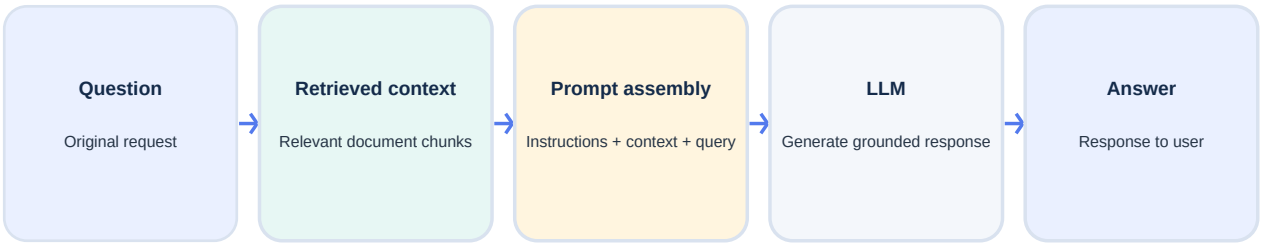
Important: The embedding model used for queries should be compatible with the one used to embed the stored chunks. Otherwise, vector similarity results may be unreliable.

Retrieval output



4. Augmentation & Answer Generation

Generation is the final stage. The LLM receives the user's question together with the retrieved context and produces a response.

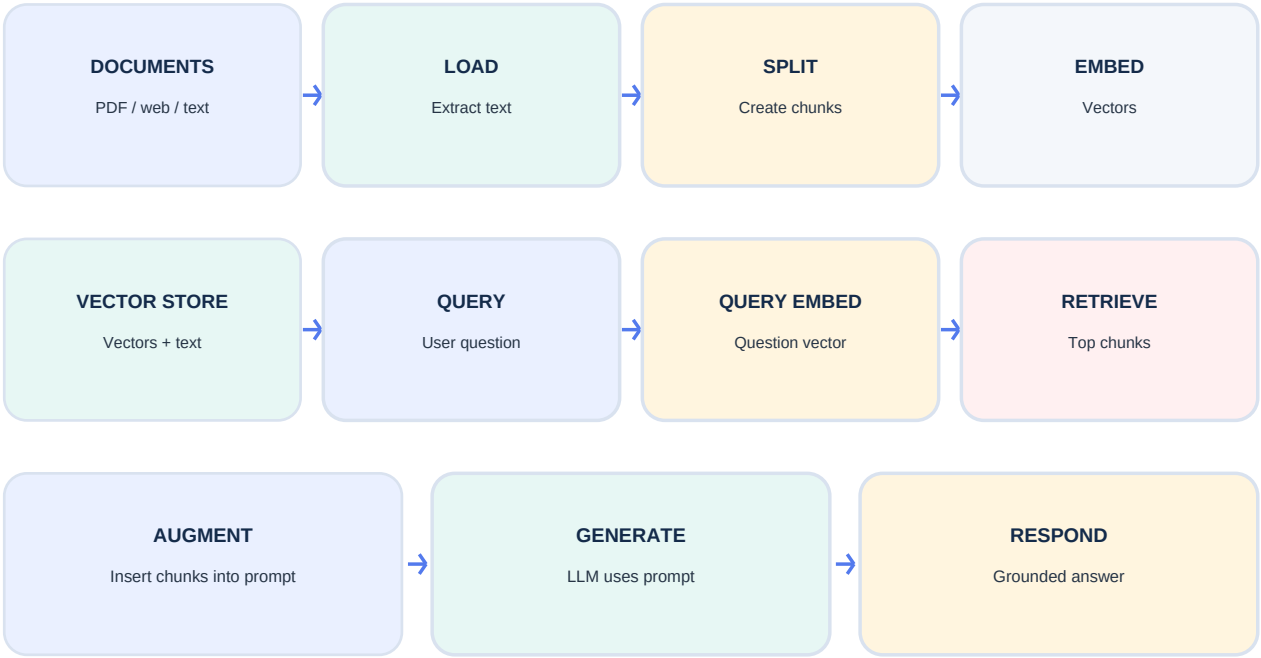


Prompt pattern

System instruction: "Answer only from the provided context. If the context is insufficient, say you don't know."

Context: ***[retrieved chunks]***
Question: ***[user question]***
Answer:

Full RAG architecture — one-page flow



The first four steps build the knowledge index; the query/retrieval/generation steps run when a user asks a question. The vector store connects these two parts.

Good RAG habits

- Keep source metadata so an answer can be traced back to a document.
- Use a clear instruction that discourages unsupported answers.

• If retrieved context does not answer the question, the system should say the evidence is insufficient rather than inventing facts.

• Check chunk quality and retrieval relevance before blaming the LLM for a poor answer.

5. Quick Revision & Glossary

Memorize the pipeline

LOAD → SPLIT → EMBED → STORE → QUERY → RETRIEVE → AUGMENT → GENERATE

Term	Simple meaning
RAG	Retrieval-Augmented Generation: retrieve external context, then generate an answer.
In-context learning	The model uses examples/information supplied in the prompt for the current task.
Document loader	Reads a source and extracts its content into document data.
Chunk	A smaller piece of a long document.
Embedding	A numerical vector representing the meaning of text.
Vector store	Stores embeddings and associated chunk information for similarity search.
Semantic search	Search based on meaning represented by vectors, not only exact words.
Retriever	Component that finds and returns relevant document chunks for a query.
Context augmentation	Adding retrieved passages to the prompt sent to the LLM.
Generation	The LLM produces the final natural-language response.

Self-check questions

1. What problem does RAG solve compared with a prompt that contains no external context?
2. Why do we split a long document into chunks?
3. What does an embedding model produce?
4. What is stored in a vector store?
5. What is the difference between retrieval and generation?
6. What should the assistant do when retrieved context is insufficient?

Answer key: 1) Supplies relevant external knowledge. 2) Makes focused passages easier to retrieve. 3) A vector. 4) Vectors linked to chunks/metadata. 5) Retrieval finds evidence; generation writes the response. 6) State that the context is insufficient / say “I don’t know.”

Source note: These notes reorganize the concepts visible in the supplied RAG.pdf into a flow-wise study guide. Some definitions are written in clearer wording to make the visible concepts easier to revise; no external research was used.